



**Facultad de  
Ciencias**  
UNAM

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

## Práctica No. 4

*Compiladores*

### Estudiante

| Nombre                          | Número de Cuenta |
|---------------------------------|------------------|
| Fuentes Ortega Diego Tonatiuh   | 320067660        |
| Carballido Camacateco Ana Lilia | 315314601        |

---

22 de marzo de 2026

---

## Validación Diego

$$\textcircled{2} F(S) = F(A) \cup F(B) \cup F(C) = \{a, b, c\}$$

$$F(A) = a$$

$$F(B) = b$$

$$F(C) = c$$

$$\textcircled{3} F(D) = F(T) \cup F(L) = \{int, float, char, id\}$$

$$F(T) = \{int, float, char\}$$

$$F(L) = \{id\} \cup F(R)$$

$$F(R) = \{, \}$$

$$\textcircled{2} F(S) = F(A) \cup F(B) = \{a, b\}$$

$$F(A) = F(a) \cup F(A) \cup F(E) \\ = \{a\} \cup \{E\}$$

$$F(B) = \{b\}$$



## Calcula el c.jto. FIRST

①

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \\ F &\rightarrow (E) \mid id \end{aligned}$$

Primero calculamos el conjunto NULL

| NULL | it. 1 | it. 2 | it. 3 |
|------|-------|-------|-------|
| E    | F     | F     | F     |
| E'   | F     | T     | T     |
| T    | F     | F     | F     |
| T'   | F     | T     | T     |
| F    | F     | F     | F     |

it. 2

$$\begin{aligned} \text{NULL}(E) &= \text{NULL}(TE') = \text{NULL}(T) \wedge \text{NULL}(E') \\ &= F \wedge F = F \end{aligned}$$

$$\begin{aligned} \text{NULL}(E') &= \text{NULL}(+TE') \vee \text{NULL}(\varepsilon) \\ &= \text{NULL}(+TE') \vee T \\ &= (\text{NULL}(+) \wedge \text{NULL}(T) \wedge \text{NULL}(E')) \vee T \\ &= (F \wedge F \wedge F) \vee T = T \end{aligned}$$

$$\begin{aligned} \text{NULL}(T) &= \text{NULL}(FT') = \text{NULL}(F) \wedge \text{NULL}(T') \\ &= F \wedge F = F \end{aligned}$$

$$\begin{aligned} \text{NULL}(T') &= \text{NULL}(*FT') \vee \text{NULL}(\varepsilon) \\ &= (\text{NULL}(*) \wedge \text{NULL}(F) \wedge \text{NULL}(T')) \vee T \\ &= (F \wedge F \wedge F) \vee T = T \end{aligned}$$

$$\begin{aligned} \text{NULL}(F) &= \text{NULL}((E)) \vee \text{NULL}(id) \\ &= (\text{NULL}((\wedge \text{NULL}(E) \wedge \text{NULL}(\wedge \vee F \\ &= (F \wedge F \wedge F) \vee F = F \end{aligned}$$



9.3

$$\text{NULL}(E) = \text{NULL}(TE') = \text{NULL}(T) \wedge \text{NULL}(E')$$

$$= F \wedge T = F$$

$$\text{NULL}(E') = \text{NULL}(+TE') \vee \text{NULL}(E)$$

$$= (\text{NULL}(+) \wedge \text{NULL}(T) \wedge \text{NULL}(E')) \vee T$$

$$= (F \wedge F \wedge T) \vee T = F \vee T = T$$

$$\text{NULL}(T) = \text{NULL}(FT') = \text{NULL}(F) \wedge \text{NULL}(T')$$

$$= F \wedge T = F$$

$$\text{NULL}(T') = \text{NULL}(*FT') \vee \text{NULL}(E)$$

$$= (\text{NULL}(*) \wedge \text{NULL}(F) \wedge \text{NULL}(T')) \vee T$$

$$= (F \wedge F \wedge T) \vee T = T$$

$$\text{NULL}(F) = \text{NULL}(E) \vee \text{NULL}(\text{id})$$

$$= (\text{NULL}(E) \wedge \text{NULL}(E) \wedge \text{NULL}(\text{id})) \vee F$$

$$= (F \wedge F \wedge F) \vee F = F$$

y tenemos

|    | NULL |
|----|------|
| E  | F    |
| E' | T    |
| T  | F    |
| T' | T    |
| F  | F    |

y ahora FIRST

| FIRST | it. 0       | it. 1       | it. 2       | it. 3       |
|-------|-------------|-------------|-------------|-------------|
| E     | $\emptyset$ | $\emptyset$ | $\emptyset$ | $\{C, id\}$ |
| E'    | $\emptyset$ | $\{+\}$     | $\{+\}$     | $\{+\}$     |
| T     | $\emptyset$ | $\emptyset$ | $\{C, id\}$ | $\{C, id\}$ |
| T'    | $\emptyset$ | $\{*\}$     | $\{*\}$     | $\{*\}$     |
| F     | $\emptyset$ | $\{C, id\}$ | $\{C, id\}$ | $\{C, id\}$ |

● it. 1

$$FIRST(E) = FIRST(TE') = FIRST(T) = \emptyset$$

$$FIRST(E') = FIRST(+TE') \cup FIRST(E)$$

$$= \{+\} \cup \emptyset = \{+\}$$

$$FIRST(T) = FIRST(FT') = FIRST(F) = \emptyset$$

$$FIRST(T') = FIRST(*FT') \cup FIRST(E)$$

$$= \{*\} \cup \emptyset = \{*\}$$

$$FIRST(F) = FIRST((E)) \cup FIRST(id)$$

$$= \{(\} \cup \{id\} = \{C, id\}$$

● it. 2

$$FIRST(E) = FIRST(TE') = FIRST(T) = \emptyset$$

$$FIRST(E') = FIRST(+TE') \cup FIRST(E)$$

$$= \{+\} \cup \emptyset = \{+\}$$

$$FIRST(T) = FIRST(FT') = FIRST(F) = \{C, id\}$$

$$FIRST(T') = FIRST(*FT') \cup FIRST(E)$$

$$= \{*\} \cup \emptyset = \{*\}$$

$$FIRST(F) = FIRST((E)) \cup FIRST(id)$$

$$= \{(\} \cup \{id\} = \{C, id\}$$



9t. 3

$$F(E) = F(TE') = F(T) = \{C, id\}$$

$$F(E') = F(+TE') \cup F(\epsilon) = \{+\} \cup \emptyset = \{+\}$$

$$F(T) = F(FT') = FIRST(F) = \{C, id\}$$

$$F(T) = F(*FT') \cup F(\epsilon) = \{*\} \cup \emptyset = \{*\}$$

$$FIRST(F) = FIRST(E) \cup FIRST(id)$$

$$= \{C\} \cup \{id\} = \{C, id\}$$

②

$$\begin{array}{lcl} S & \longrightarrow & ABC \\ A & \longrightarrow & a | \epsilon \\ B & \longrightarrow & b | \epsilon \\ C & \longrightarrow & c \end{array}$$

Como esta gramática es más sencilla nos ahorraremos sacar el NULL

$$FIRST(S) = FIRST(ABC) \quad // A anulable$$

$$= FIRST(A) \cup FIRST(B) \cup FIRST(c)$$

$$= (\{a\} \cup \emptyset) \cup (\{b\} \cup \emptyset) \cup \{c\}$$

$$= \{a, b, c\}$$

③

$$\begin{array}{lcl} D & \longrightarrow & TL \\ T & \longrightarrow & int | float | char \\ L & \longrightarrow & id R \\ R & \longrightarrow & , L | \epsilon \end{array}$$

Igualmente nos ahorraremos sacar NULL

$$FIRST(D) = FIRST(TL) = FIRST(T)$$

$$= (FIRST(int) \cup FIRST(float) \cup FIRST(char))$$

$$= \{int, float, char\}$$

$$FIRST(L) = FIRST(idR) = \{id\}$$

$$FIRST(R) = FIRST(,L) \cup FIRST(\epsilon) = \{, \}$$



## Calcula el cjo FOLLOW

$$\begin{array}{lcl}
 E & \rightarrow & TE' \$ \\
 E' & \rightarrow & + TE' \mid \epsilon \\
 T & \rightarrow & FT' \\
 T' & \rightarrow & * FT' \mid \epsilon \\
 F & \rightarrow & (E) \mid id
 \end{array}$$

Con NULL y FIRST calculamos FOLLOW

$$F \rightarrow (E)$$

$$FOLLOW(E) = \{ \$ \} \cup \{ ) \} = \{ \$, ) \}$$

$$E \rightarrow TE', \quad E' \rightarrow + TE'$$

$$\begin{aligned}
 FOLLOW(E') &= FOLLOW(E) \cup FOLLOW(E') \\
 &= \{ \$, ) \}
 \end{aligned}$$

$$E \rightarrow TE', \quad E' \rightarrow + TE'$$

$$\begin{aligned}
 FOLLOW(T) &= FIRST(E') \cup FOLLOW(E) \cup FOLLOW(E') \\
 &= \{ + \} \cup \{ \$, ) \} = \{ +, \$, ) \}
 \end{aligned}$$

$$T \rightarrow FT', \quad T' \rightarrow * FT'$$

$$\begin{aligned}
 FOLLOW(T') &= FOLLOW(T) \cup FOLLOW(T') \\
 &= \{ +, \$, ) \}
 \end{aligned}$$

$$T \rightarrow FT', \quad T' \rightarrow * FT'$$

$$\begin{aligned}
 FOLLOW(F) &= FIRST(T') \cup FOLLOW(T) \cup FOLLOW(T') \\
 &= \{ * \} \cup \{ +, \$, ) \} = \{ *, +, \$, ) \}
 \end{aligned}$$



$$\textcircled{2} \quad \begin{array}{lcl} S & \longrightarrow & AB\$ \\ A & \longrightarrow & aA \mid \epsilon \\ B & \longrightarrow & b \mid \epsilon \end{array}$$

Calculamos el Cjto FIRST

$$\begin{aligned} \text{FIRST}(S) &= \text{FIRST}(AB) = \text{FIRST}(A) \cup \text{FIRST}(B) \\ &= \{a\} \cup \{b\} = \{a, b\} \end{aligned}$$

$$\begin{aligned} \text{FIRST}(A) &= \text{FIRST}(aA) \cup \text{FIRST}(\epsilon) \\ &= \{a\} \cup \emptyset = \{a\} \end{aligned}$$

$$\begin{aligned} \text{FIRST}(B) &= \text{FIRST}(b) \cup \text{FIRST}(\epsilon) \\ &= \{b\} \cup \emptyset = \{b\} \end{aligned}$$

$$\text{FOLLOW}(S) = \{\$ \}$$

$$S \longrightarrow AB, \quad A \longrightarrow aA$$

$$\begin{aligned} \text{FOLLOW}(A) &= \text{FIRST}(B) \cup \text{FOLLOW}(S) \cup \text{FOLLOW}(A) \\ &= \{b\} \cup \{\$ \} = \{b, \$ \} \end{aligned}$$

$$S \longrightarrow AB$$

$$\text{FOLLOW}(B) = \text{FOLLOW}(S) = \{\$ \}$$



$$S \rightarrow (L) \$$$

$$3 \quad L \rightarrow SL'$$

$$L' \rightarrow , SL' \mid t$$

Conjunto FIRST

$$\text{FIRST}(S) = \text{FIRST}((L)) = \{( \}$$

$$\text{FIRST}(L) = \text{FIRST}(SL') = \text{FIRST}(S) = \{( \}$$

$$\text{FIRST}(L') = \text{FIRST}(, SL') \cup \text{FIRST}(t)$$

$$= \{ , \} \cup \{ t \} = \{ , , t \}$$

Conjunto FOLLOW

$$L \rightarrow SL', L' \rightarrow , SL'$$

$$\text{FOLLOW}(S) = \text{FIRST}(L') \cup \text{FOLLOW}(L) \cup \text{FOLLOW}(L')$$

$$= \{ , , \$ \} \cup \text{FOLLOW}(L) \cup \text{FOLLOW}(L')$$

$$S \rightarrow (L \_)$$

$$\text{FOLLOW}(L) = \{ ) \}$$

$$L \rightarrow SL', L' \rightarrow , SL'$$

$$\text{FOLLOW}(L') = \text{FOLLOW}(L) \cup \text{FOLLOW}(L')$$

$$= \{ ) \}$$

$$\text{FOLLOW}(S) = \{ , , \$ , ) \}$$

$$\text{Follow}(E) = \{\$, \epsilon\}$$

$$\text{Follow}(E') = \{id, ($$

$$1. E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

$$T \in \{a, b, c, \epsilon\}$$

$$\text{Ter} \{id, (, ), +, *, \epsilon\}$$

$$\text{First}(E) = \{id, ( \}$$

$$\text{First}(E') = \{+, \epsilon\}$$

$$\text{First}(T) = \{id, ( \}$$

$$\text{First}(T') = \{*, \epsilon\}$$

$$\text{First}(F) = \{id, ( \}$$

$$\text{Follow}(E) = \{), \$\}$$

$$\text{Follow}(E') = \{), \$\}$$

$$\text{Follow}(T) = \{+, \$\}$$

$$\text{Follow}(T') = \{+, \$\}$$

$$\text{Follow}(F) = \{*, \$\}$$

\*2-

$$\underline{S} \rightarrow \underline{A} \underline{B} \underline{C} \quad \$$$

$$\underline{A} \rightarrow a \mid \epsilon$$

$$\underline{B} \rightarrow b \mid \epsilon$$

$$\underline{C} \rightarrow c$$

$$\text{First}(S) = \{a, \epsilon\}$$

$$\text{First}(A) = \{a, \epsilon\}$$

$$\text{First}(B) = \{b, \epsilon\}$$

$$\text{First}(C) = \{c\}$$

$$S = \$$$

$$A = b, c$$

$$B = c$$

$$C = \$$$

agrega first(S)



3.  $D \rightarrow \underline{T} \underline{L}$

$\underline{T} \rightarrow \text{int} \mid \text{float} \mid \text{char}$

$\underline{L} \rightarrow \text{id} \underline{R}$

$R \rightarrow , \underline{L} \mid \epsilon$

$\text{First}(D) = \{\text{int}, \text{float}, \text{char}\}$

$\text{First}(T) = \{\text{int}, \text{float}, \text{char}\}$

$\text{First}(L) = \{\text{id}\}$

$\text{First}(R) = \{, \mid \epsilon\}$

4

$\underline{X} \rightarrow \underline{Y} \underline{Z} \mid a$

$Y \rightarrow b \mid \epsilon$

$Z \rightarrow c \mid \epsilon$

$\text{First}(X) = \{a, b\}$

$\text{First}(Y) = \{b, \epsilon\}$

$\text{First}(Z) = \{c, \epsilon\}$

\* FOLLOW

$S \rightarrow (L)$

$L \rightarrow SL'$

$L' \rightarrow , SL' | \epsilon$

$\text{First}(S) = \{ ( \}$

$\text{First}(L) = \{ ( \}$

$\text{First}(L') = \{ , , \epsilon \}$

$\text{FOLLOW}(S) = \{ , , \$ , ) \}$

$\text{FOLLOW}(L) = \{ ) \}$

$\text{FOLLOW}(L') = \{ ) \}$



## Descripción de implementación

### Estructuras de datos

#### Símbolo (symbol)

Cada símbolo del vocabulario de la gramática se representa con tres campos:

- **symbol**: puntero a la cadena de texto del símbolo, duplicada con **strdup** para que cada instancia tenga su propia memoria.
- **symbol\_length**: longitud precalculada de la cadena, evitando llamadas repetidas a **strlen**.
- **is\_terminal**: booleano que indica si el símbolo es terminal (**true**) o no terminal (**false**).

#### Producciones

Dentro de cada producción, los símbolos se identifican con un entero:

$$\text{id} = \begin{cases} t & \text{si es terminal, } t \in [0, T - 1] \\ T + A & \text{si es no terminal, } A \in [0, N - 1] \end{cases}$$

donde  $T$  es el número de terminales y  $N$  el de no terminales. Para conocer el índice real de un no terminal necesitamos restar  $T$ :  $X_i = \text{sym\_id} - T$ .

#### Tabla FIRST

La tabla FIRST la almacenamos como un arreglo de booleanos de tamaño  $N \times T$ , indexado como:

$$\text{first\_table}[A \times T + t]$$

El valor es **true** si el terminal  $t$  pertenece a  $\text{FIRST}(A)$ .

#### Vector nullable

Arreglo de booleanos de tamaño  $N$  donde **nullable[A] = true** indica que el no terminal  $A$  puede derivar en la cadena vacía  $\varepsilon$ . Esta separado de **first\_table** porque su semántica es distinta, no es un terminal que aparezca en FIRST, más como una propiedad del no terminal.

#### Función find\_terminal\_id

Recorremos linealmente **g->terminals**.

#### Función add\_symbol\_to\_array

Lo primero que hace la función es verificar que ninguno de los tres punteros recibidos sea NULL.

El arreglo de símbolos no tiene tamaño fijo, crece de uno en uno conforme se agregan elementos. Así que se llama a **realloc(\*arr, (\*count + 1) \* sizeof(symbol))** para solicitar un bloque de memoria que contenga un elemento más que el actual.

El resultado se guarda en una variable local en lugar de sobrescribir **\*arr** directamente ya que si **realloc** falla y devuelve NULL, el puntero original **\*arr** no se habrá modificado y el "llamador" mantiene acceso al bloque previo. Solamente cuando **realloc** tiene éxito se actualiza **\*arr = new\_array**.

Ya que el arreglo creció, se duplica la cadena **text** con **strdup**. Al duplicar la cadena, cada entrada del arreglo es dueña de su propia copia y puede liberarse de forma independiente en **free\_symbol\_array**.

Con la cadena duplicada disponible, se rellenan los tres campos de la nueva entrada **(\*arr)[\*count]**:

- `symbol` recibe el puntero a la cadena duplicada.
- `symbol_length` se calcula con `strlen` y se guarda para evitar recalcularlo.
- `is_terminal` se copia directamente del parámetro, registrando si el símbolo es terminal o no.

El contador `*count` se incrementa al final.

### Función `compute_first_tables`

La función itera sobre todas las producciones repetidamente hasta que ninguna iteración agrega información nueva. Para cada producción  $A \rightarrow X_1 X_2 \dots X_n$ :

1. Se recorren los  $X_i$  de izquierda a derecha.
2. Si  $X_i$  es terminal, se agrega a  $\text{FIRST}(A)$  y se detiene el recorrido.
3. Si  $X_i$  es no terminal, se propagan todos los terminales de  $\text{FIRST}(X_i)$  hacia  $\text{FIRST}(A)$ , excluyendo `epsilon_id`. Si  $X_i$  no es nullable, se detiene el recorrido.
4. Si todos los  $X_i$  resultaron nullable, se marca `nullable[A] = true`.

El símbolo `epsilon` se excluye de la propagación entre no terminales. Su información se captura únicamente a través de `nullable`, si  $A$  es nullable, el "llamador" puede agregar `epsilon` a  $\text{FIRST}(A)$  al momento. Así evitamos que `epsilon` se propague como terminal ordinario.

### Función `compute_follow_table`

Esta función construye la tabla FOLLOW para los símbolos no terminales de la gramática. Esta tabla se representa como una matriz con dimensión  $N \times (T + 1)$ , donde  $N$  corresponde al número de no terminales y  $T$  al número de terminales, agregando una columna adicional para el símbolo de fin de cadena `$`.

1. Comienza con el FOLLOW de inicio  $\text{FOLLOW}(\text{start}) = \$$ .
2. Se recorren las producciones  $\text{FIRST}(\beta)$  a  $\text{FOLLOW}(B)$ .
3. Se agrega  $\text{FIRST}(\beta)$  a  $\text{FOLLOW}(B)$ .
4. Si  $\beta$  es nullable, entonces se agrega  $\text{FOLLOW}(A)$  a  $\text{FOLLOW}(B)$ .
5. Se repite hasta que no hayan cambios.

### Función `collect_first_for_non_terminal`

En esta función se recorre la fila correspondiente al no terminal en la tabla FIRST y se agrega en un arreglo de salida, que representa el conjunto FIRST del no terminal, cada uno de los terminales que tengan valor True. Luego, si el símbolo terminal no es anulable, se agrega el símbolo  $\epsilon$  al arreglo.

### Función `collect_follow_for_non_terminal`

Esta función extrae el conjunto FOLLOW de un símbolo no terminal a partir de la tabla FOLLOW. Se recorre la fila correspondiente al no terminal dentro de la tabla y se agregan al arreglo de salida todos los terminales presentes. En caso de que la última columna de la tabla esté activada, se añade el símbolo `$`, indicando el final de la cadena. Devuelve un arreglo con todos los símbolos del conjunto FOLLOW del símbolo terminal.

### Función `compute_first_for_non_terminal`

La función obtiene el conjunto FIRST del símbolo terminal. Primero se validan los datos de entrada y después se construyen las tablas FIRST y nullable usando las funciones `compute_first_tables` y `collect_first_for_non_terminal` para obtener el conjunto correspondiente. Devuelve el número de símbolos obtenidos del conjunto FIRST.



### Función `compute_follow_for_non_terminal`

Primero se calculan las tablas FIRST y nullable, necesarias para la propagación de FOLLOW. Luego, se construye la tabla FOLLOW usando las funciones `compute_follow_table` y se extrae el conjunto correspondiente usando la función `collect_follow_for_non_terminal`.  
Devuelve un arreglo con el conjunto FOLLOW del símbolo.

### Función `compute_first_for_start_symbol`

Esta función obtiene el conjunto FIRST del símbolo inicial utilizando la función `compute_first_for_non_terminal` con el índice cero.

### Función `compute_follow_for_start_symbol`

Esta función obtiene el conjunto FOLLOW del símbolo inicial utilizando la función `compute_follow_for_non_terminal` con el índice cero.

### Función `free_symbol_array`

Se encarga de liberar el espacio de memoria creados cuando se calcula FIRST y FOLLOW los cuales generan arreglos. Primero se libera la memoria asociada a cada símbolo y después el arreglo.

## Dificultades encontradas

Durante la implementación no hubo demasiadas complicaciones más que errores simples de null, por escribir algo mal o confundir idiomas, donde tuve más problemas fue al entender el conjunto first y follow, pero eso se arregló bastante al repasar y pasar a limpio las actividades de laboratorio.

## Resultados obtenidos

```
→ docker run --rm first_and_follow-validator
=====
Validador P4 - FIRST/FOLLOW
=====
Project: /first_and_follow
Build dir: /first_and_follow/build_validator
-----
Step 0: Build first_and_follow [PASS]
  Executable generated: /first_and_follow/build_validator/first_and_follow
-----
Step 1: Validate case: Nullable prefix before terminal [PASS]
  Case 'Nullable prefix before terminal' passed.
-----
Step 2: Validate case: Left recursion with terminal tail [PASS]
  Case 'Left recursion with terminal tail' passed.
-----
Step 3: Validate case: Two nullable symbols in sequence [PASS]
  Case 'Two nullable symbols in sequence' passed.
-----
Step 4: Validate case: Recursive nullable non-terminal [PASS]
  Case 'Recursive nullable non-terminal' passed.
-----
Summary: 5/5 steps passed
```